

```
400, "publication": "/api/v1/publications/3/")' -i http://localhost:8000/api/v1/books/
```

To use filters, add fields in resources to allow filters on those fields:

```
class PublicationResource(ModelResource):
    """Resource for Publication
    """

    class Meta:
        queryset = Publication.objects.all()
        resource_name = 'publications'
        authorization = Authorization()
        filtering = {
            'name': ['exact', 'contains']
        }

class BookResource(ModelResource):
    """Resource for Book
    """

    publication = fields.ForeignKey(PublicationResource,
        'publication')

    class Meta:
        queryset = Book.objects.all()
        resource_name = 'books'
        authorization = Authorization()
        filtering = {
            'publication': ALL_WITH_RELATIONS,
            'price': ['exact', 'lt', 'lte', 'gte', 'gt'],
        }
```

Now filter based on price

```
Curl http://localhost:8000/api/v1/books/?price=450
```

This will return the books object with a price= ₹450.

```
{"meta": {"limit": 20, "next": null, "offset": 0, "previous": null, "total_count": 1}, "objects": [{"author": "Arthur Conan Doyle", "description": "The great detective series", "id": 2, "name": "The Memoirs of Sherlock Holmes", "price": 450, "publication": "/api/v1/publications/1/", "resource_uri": "/api/v1/books/2/"}]}
```

If we want all the books priced below ₹ 450, then use the "lt" keyword as follows:

```
curl http://localhost:8000/api/v1/books/?price_lt=450
{"meta": {"limit": 20, "next": null, "offset": 0, "previous": null, "total_count": 2}, "objects": [{"author": "Paulo Coelho", "description": "The Alchemist is a novel by Brazilian author Paulo Coelho which was first published in 1988.", "id": 1, "name": "The Alchemist", "price": 350, "publication": "/api/v1/publications/1/", "resource_uri": "/api/v1/books/1/"}, {"author": "Amish Tripathi",
```

```
"description": "The shiva trilogy 1", "id": 3, "name": "The Immortals of Meluha", "price": 400, "publication": "/api/v1/publications/1/", "resource_uri": "/api/v1/books/3/"}]}
```

Filter based on the publication's name is done the same way that we filter for the name field by adding it to resource filters.

## Updating resources

To update the resource, we just need to provide fields to be updated for a specific resource.

```
curl -H "Content-Type: application/json" -X PATCH -d '{"name": "The Nagas"}' -i http://localhost:8000/api/v1/books/3/
```

Now get the resource using the GET method. Then the output should contain the updated books' names.

## Deleting resources

To delete the particular resource, use the DELETE method of REST.

```
curl -H "Content-Type: application/json" -X DELETE -i http://localhost:8000/api/v1/books/3/
```

Now, when you GET the book with ID 3, the 404 status should be returned.

```
curl -H -i http://localhost:8000/api/v1/books/3/
```

To summarise, Tastypie is a great framework for implementing the REST API in Django. A major advantage is the great community that stands behind it, resulting in good documentation and it being well tested. In this article, we looked at the basic CRUD operation on resources, and we can now implement custom authentication and authorisation, override the method of resource classes, and implement our own method as per our requirements. [END](#) 

## References

- [1] <http://django-tastypie.readthedocs.io/en/latest/index.html>
- [2] <http://django-tastypie.readthedocs.io/en/latest/authentication.html>
- [3] <http://django-tastypie.readthedocs.io/en/latest/authorization.html>
- [4] <https://docs.djangoproject.com/en/1.10/>

## By: Yogesh Kamble

The author is currently working at Synerzip Softtech Pvt Ltd as senior software developer. He is interested in working on back-end-technologies and loves to try out new Linux distros. He can be contacted at [yogesh.kamble@aol.com](mailto:yogesh.kamble@aol.com)